Phase 10: Generating Trees from Traversals

0. Purpose of This Phase

Phase 10 teaches how to **rebuild a binary tree when traversal orders are given**.

In earlier phases, you usually started with a tree and then found its traversal.

Example:

```
Given tree  -> find preorder
Given tree  -> find inorder
Given tree  -> find postorder
```

In Phase 10, the direction is reversed.

Now you are given traversal arrays, and you must rebuild the original tree.

Example:

```
Given preorder and inorder -> rebuild the binary tree
```

The main beginner idea is:

```
Traversal arrays are clues.
We use those clues to reconstruct the tree.
```

This phase is the final phase because it depends on many earlier topics:

- binary tree structure
- left and right child pointers
- preorder traversal
- inorder traversal
- postorder traversal
- recursion
- subtree splitting
- linked node construction
- nullptr base cases
- time and space complexity

---

1. Current Progress Analysis

Before learning Phase 10, you should already understand the ideas from Phases 1 to 9.

Previous topic	Why it matters in Phase 10
Root	Traversal reconstruction starts by finding the root.
Left child and right child	We must rebuild the left and right subtrees.
Leaf node	Leaf nodes become base cases in recursion.
Subtree	Phase 10 repeatedly splits the tree into subtrees.
Preorder traversal	Preorder helps us find the root early.
Inorder traversal	Inorder helps us split left and right subtrees.
Postorder traversal	Postorder can also help us find the root, but from the end.
Recursion	Tree reconstruction is naturally recursive.
Linked representation	We create nodes and connect them using pointers.
nullptr	Empty subtree positions return nullptr.

So your current position is:

```
You already know how to traverse a tree.
Now you are learning how to build a tree from traversal results.
```

Important bridge:

```
Before Phase 10:
Tree -> Traversal

In Phase 10:
Traversal -> Tree
```

---

2. What Phase 10 Covers

Phase 10 covers these main ideas:

1. Why one traversal is not enough to build a unique tree.
2. Why preorder alone cannot reconstruct a unique binary tree.
3. Why postorder alone cannot reconstruct a unique binary tree.

4. Why preorder plus postorder is usually not enough for ordinary binary trees.
5. Why inorder is very important.
6. How preorder + inorder can rebuild a unique binary tree.
7. How inorder + postorder can rebuild a unique binary tree.
8. How recursion is used in tree reconstruction.
9. How to split the inorder array into left and right subtrees.
10. How to write C++ code for preorder + inorder reconstruction.
11. Why a map improves the time complexity.
12. How to dry run the reconstruction process.
13. Common beginner mistakes.
14. Final memory tricks.

Part A: The Big Idea

3. What Does “Generating a Tree from Traversals” Mean?

Generating a tree from traversals means:

```
Use traversal arrays to rebuild the original binary tree.
```

A traversal is only an order of visiting nodes.

For example:

```
Preorder: A B C
Inorder:  B A C
```

These arrays do not directly show the tree picture. But if we understand how preorder and inorder work, we can reconstruct the tree.

The tree is:

```
   A
  /
 B   C
```

So Phase 10 is like solving a puzzle.

The traversal arrays are the clues.

The tree is the final answer.

4. Review of Traversal Rules

Before reconstructing trees, you must remember the traversal rules.

Traversal	Rule	Simple meaning
Preorder	Root, Left, Right	Visit root first.
Inorder	Left, Root, Right	Visit root in the middle.
Postorder	Left, Right, Root	Visit root last.

Memory tricks:

```
Preorder  = root before children
Inorder   = root between left and right
Postorder = root after children
```

5. Why This Phase Is Harder Than Normal Traversal

Normal traversal is easier because the tree already exists.

Example:

```
   A
  /
 B   C
```

If we need preorder, we can simply visit:

```
A B C
```

But in Phase 10, we are not given the tree picture.

We may only be given:

```
Preorder: A B C
Inorder:  B A C
```

From this, we must figure out:

```
   A
  /
 B   C
```

So Phase 10 requires two skills:

1. Understanding traversal order.
2. Reconstructing the tree shape using recursion.

Part B: Why One Traversal Is Not Enough

6. Can One Traversal Build a Unique Tree?

No.

One traversal does not give enough information.

A single traversal tells us only the visiting order. It does not always tell us which node is on the left side and which node is on the right side.

---

7. Example: Preorder Alone Is Not Enough

Suppose we are given:

```
Preorder: A B
```

Preorder means:

```
Root first, then left, then right
```

So we know:

```
A is the root.
B comes after A.
```

But we do not know whether B is the left child or the right child.

Tree 1:

```
A
/
B
```

Tree 2:

```
A

  B
```

Both trees have the same preorder:

```
A B
```

So preorder alone cannot uniquely rebuild the tree.

---

8. Example: Inorder Alone Is Not Enough

Suppose we are given:

```
Inorder: A B
```

Inorder means:

```
Left, Root, Right
```

But we do not know whether:

- A is the root and B is the right child, or
- B is the root and A is the left child.

Tree 1:

```
A

  B
```

Inorder:

```
A B
```

Tree 2:

```
  B
 /
A
```

Inorder:

```
A B
```

So inorder alone is also not enough.

---

9. Example: Postorder Alone Is Not Enough

Suppose we are given:

```
Postorder: B A
```

Postorder means:

```
Left, Right, Root
```

So we know A is probably the root because postorder prints root last.

But B could still be the left child or the right child.

Tree 1:

```
A
/
B
```

Postorder:

```
B A
```

Tree 2:

```
A

  B
```

Postorder:

```
B A
```

So postorder alone is not enough.

---

10. Key Rule About One Traversal

A single traversal is not enough to uniquely generate an ordinary binary tree.

Memory trick:

```
One traversal gives order.
But one traversal does not give enough shape information.
```

Part C: Is Preorder + Postorder Enough?

11. Preorder + Postorder Usually Is Not Enough

Preorder tells us the root early.

```
Preorder = Root, Left, Right
```

Postorder tells us the root at the end.

```
Postorder = Left, Right, Root
```

But for ordinary binary trees, preorder + postorder still may not clearly tell us which nodes belong to the left subtree and which nodes belong to the right subtree.

---

12. Example: Preorder + Postorder Ambiguity

Suppose:

```
Preorder:  A B
Postorder: B A
```

This could be:

```
A
/
B
```

Or:

```
A

  B
```

Both have:

```
Preorder:  A B
Postorder: B A
```

So preorder + postorder usually does not uniquely reconstruct an ordinary binary tree.

Important note:

```
Preorder + postorder can sometimes work under special restrictions,
for example with full/strict binary trees.
But for ordinary binary trees, it is usually not enough.
```

Part D: Why Inorder Is Important

13. Inorder Is the Splitter

Inorder is the most important traversal for reconstruction.

Why?

Because inorder has this rule:

```
Inorder = Left, Root, Right
```

That means if we know the root, we can find the root inside the inorder array.

Then:

```
Everything before the root = left subtree
Everything after the root  = right subtree
```

This is the key idea of Phase 10.

---

14. Simple Inorder Splitting Example

Suppose the inorder traversal is:

```
D B E A F C G
```

Suppose we already know the root is:

```
A
```

Now find A in inorder:

```
D B E   A   F C G
```

Split around A:

```
Left subtree nodes  = D B E
Root                = A
Right subtree nodes = F C G
```

So we know the tree has this rough shape:

```
       A
      /
 D B E     F C G
```

This does not mean D B E are directly connected as shown. It only means those nodes belong somewhere in the left subtree.

And F C G belong somewhere in the right subtree.

---

15. Why Inorder Must Be Paired With Another Traversal

Inorder can split the tree only if we already know the root.

But inorder alone does not always tell us the root.

So we pair inorder with:

```
Preorder
```

or:

```
Postorder
```

Preorder helps because its first element is the root.

Postorder helps because its last element is the root.

So:

```
Preorder + Inorder  -> unique tree, if values are unique
Inorder + Postorder -> unique tree, if values are unique
```

---

16. The Most Important Memory Trick

```
Preorder tells WHO the root is.
Inorder tells WHERE the left and right subtrees are.
```

Another version:

```
Preorder gives the root.
Inorder splits the tree.
```

For postorder:

```
Postorder gives the root from the end.
Inorder splits the tree.
```

Part E: Preorder + Inorder Reconstruction

17. Preorder + Inorder Rules

To reconstruct a binary tree from preorder and inorder, remember these traversal rules:

```
Preorder = Root, Left, Right
Inorder  = Left, Root, Right
```

Preorder helps us find the root.

Inorder helps us split the left and right subtrees.

---

18. Golden Rule for Preorder + Inorder

Use this process:

1. Take the first unused value from preorder.
2. That value is the root of the current subtree.
3. Find that root value in inorder.
4. Values on the left side of the root in inorder belong to the left subtree.
5. Values on the right side of the root in inorder belong to the right subtree.
6. Recursively repeat the same process for the left and right subtrees.

Short version:

```
Read root from preorder.
Split using inorder.
Repeat recursively.
```

---

19. Beginner Example 1

Given:

```
Preorder: A B C
Inorder:  B A C
```

Step 1: Find root from preorder.

```
Preorder starts with A.
So root = A.
```

Step 2: Find A in inorder.

```
Inorder: B A C
```

Split around A:

```
Left subtree  = B
Root          = A
Right subtree = C
```

Step 3: Build the tree.

```
   A
  /
 B   C
```

Step 4: Check preorder.

```
Root, Left, Right = A B C
```

Correct.

Step 5: Check inorder.

```
Left, Root, Right = B A C
```

Correct.

---

20. Beginner Example 2

Given:

```
Preorder: A B D E C
Inorder:  D B E A C
```

Step 1: First preorder value is root.

```
Root = A
```

Step 2: Split inorder around A.

```
D B E   A   C
```

So:

```
Left subtree nodes  = D B E
Right subtree nodes = C
```

Current shape:

```
      A
     /
 D B E   C
```

Step 3: Move to next preorder value.

Preorder after A:

```
B D E C
```

The next root is B.

So B is the root of the left subtree.

Step 4: Split left inorder part around B.

Left inorder part:

```
D B E
```

Split around B:

```
D   B   E
```

So:

```
Left child of B  = D
Right child of B = E
```

Step 5: Right subtree of A is C.

Final tree:

```
     A
    /
   B   C
  /
 D   E
```

Check preorder:

```
A B D E C
```

Check inorder:

```
D B E A C
```

Both match.

Part F: Full Dry Run from the Curriculum

21. Given Traversals

We are given:

```
Preorder: 4, 7, 9, 6, 3, 2, 5, 8, 1
Inorder:  7, 6, 9, 3, 4, 5, 8, 2, 1
```

Goal:

```
Rebuild the original binary tree.
```

---

22. Step 1: First Root

Preorder starts with:

```
4
```

So:

```
Root = 4
```

Find 4 in inorder:

```
7, 6, 9, 3, 4, 5, 8, 2, 1
```

Split around 4:

```
Left subtree nodes  = 7, 6, 9, 3
Root                = 4
Right subtree nodes = 5, 8, 2, 1
```

Current tree:

```
       4
      /
  left   right
```

---

23. Step 2: Build the Left Subtree of 4

The left subtree contains these inorder nodes:

```
7, 6, 9, 3
```

Now look at preorder after 4:

```
7, 9, 6, 3, 2, 5, 8, 1
```

The next root is:

```
7
```

So 7 is the root of the left subtree of 4.

Tree so far:

```
       4
      /
     7
```

Find 7 in the left inorder part:

```
7, 6, 9, 3
```

Split around 7:

```
Left of 7  = empty
Right of 7 = 6, 9, 3
```

So 7 has no left child.

Its right subtree contains:

```
6, 9, 3
```

---

24. Step 3: Build the Right Subtree of 7

Next preorder value is:

```
9
```

So 9 becomes the right child of 7.

Tree so far:

```
      4
     /
    7

      9
```

The inorder part for this subtree is:

```
6, 9, 3
```

Split around 9:

```
Left of 9  = 6
Right of 9 = 3
```

So:

```
6 is in the left subtree of 9.
3 is in the right subtree of 9.
```

Tree so far:

```
        4
       /
      7

        9
       /
      6   3
```

---

25. Step 4: Build the Right Subtree of 4

The right subtree of 4 contains these inorder nodes:

```
5, 8, 2, 1
```

The next preorder value after finishing the left side is:

```
2
```

So 2 becomes the right child of 4.

Tree so far:

```
        4
       /
      7   2

        9
       /
      6   3
```

Find 2 in the right inorder part:

```
5, 8, 2, 1
```

Split around 2:

```
Left of 2  = 5, 8
Right of 2 = 1
```

So:

```
Left subtree of 2 contains 5 and 8.
Right subtree of 2 contains 1.
```

---

26. Step 5: Build the Left Subtree of 2

Next preorder value is:

```
5
```

So 5 becomes the left child of 2.

Tree so far:

```
        4
       /
      7   2
       \ /
        9 5
       /
      6   3
```

The inorder part for this subtree is:

```
5, 8
```

Find 5 in it:

```
5, 8
```

Split around 5:

```
Left of 5  = empty
Right of 5 = 8
```

So 5 has no left child, and 8 is in its right subtree.

Next preorder value is:

```
8
```

So 8 becomes the right child of 5.

Tree so far:

```
      4
     /
    7   2
     \ /
      9 5
     / \
    6  3 8
```

---

27. Step 6: Build the Right Subtree of 2

The right subtree of 2 contains:

```
1
```

Next preorder value is:

```
1
```

So 1 becomes the right child of 2.

Final tree:

```
      4
     /
    7   2
     \ /
      9 5 1
     / \
    6  3 8
```

---

28. Final Tree Check

Final tree:

```
      4
     /
    7   2
     \ /
      9 5 1
     / \
    6  3 8
```

Preorder check:

```
Root, Left, Right
```

Visit:

```
4, 7, 9, 6, 3, 2, 5, 8, 1
```

This matches the given preorder.

Inorder check:

```
Left, Root, Right
```

Visit:

```
7, 6, 9, 3, 4, 5, 8, 2, 1
```

This matches the given inorder.

So the generated tree is correct.

Part G: Recursive Algorithm

29. Why Recursion Is Used

Tree reconstruction is naturally recursive because a tree is made of subtrees.

At any node, we can say:

```
Build the root.
Build the left subtree.
Build the right subtree.
Connect them together.
```

That is recursive thinking.

---

30. Recursive Function Meaning

We can imagine a function like this:

```
buildTree(preorder, inorder range)
```

The function means:

```
Build the tree whose nodes are inside this inorder range.
```

At each recursive call:

1. Pick the next root from preorder.
2. Create a node for that root.
3. Find the root inside inorder.
4. Build the left subtree using the left part of inorder.
5. Build the right subtree using the right part of inorder.
6. Return the root node.

---

31. Base Case

The base case is:

```
If the inorder range is empty, return nullptr.
```

In code, this often looks like:

```
if (inStart > inEnd) {
    return nullptr;
}
```

Simple meaning:

```
There are no nodes in this subtree.
So this child pointer should be nullptr.
```

---

32. Recursive Pattern

The general recursive pattern is:

```
If no nodes exist:
    return nullptr

Pick root
Create root node
Split inorder
root->left = build left subtree
root->right = build right subtree
Return root
```

This is very similar to earlier recursive tree functions, but now instead of only reading a tree, we are creating one.

Part H: C++ Code for Preorder + Inorder Reconstruction

33. Complete Optimized C++ Program

```cpp
#include <iostream>
#include <vector>
#include <unordered_map>
using namespace std;

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
        data = val;
        left = nullptr;
        right = nullptr;
    }
};

Node* buildTreeHelper(
    const vector<int>& preorder,
    int& preIndex,
    int inStart,
    int inEnd,
    unordered_map<int, int>& inorderIndex
) {
    // Base case: no nodes in this subtree
    if (inStart > inEnd) {
        return nullptr;
    }

    // Preorder gives the next root
    int rootValue = preorder[preIndex];
    preIndex++;

    Node* root = new Node(rootValue);

    // Find root inside inorder
    int split = inorderIndex[rootValue];

    // Build left subtree
    root->left = buildTreeHelper(
```

```
        preorder,
        preIndex,
        inStart,
        split - 1,
        inorderIndex
    );

    // Build right subtree
    root->right = buildTreeHelper(
        preorder,
        preIndex,
        split + 1,
        inEnd,
        inorderIndex
    );

    return root;
}

Node* buildTree(const vector<int>& preorder, const vector<int>& inorder) {
    unordered_map<int, int> inorderIndex;

    for (int i = 0; i < inorder.size(); i++) {
        inorderIndex[inorder[i]] = i;
    }

    int preIndex = 0;

    return buildTreeHelper(
        preorder,
        preIndex,
        0,
        inorder.size() - 1,
        inorderIndex
    );
}

void preorderPrint(Node* root) {
    if (root == nullptr) return;

    cout << root->data << " ";
    preorderPrint(root->left);
    preorderPrint(root->right);
}

void inorderPrint(Node* root) {
    if (root == nullptr) return;
```

```
    inorderPrint(root->left);
    cout << root->data << " ";
    inorderPrint(root->right);
}

void postorderPrint(Node* root) {
    if (root == nullptr) return;

    postorderPrint(root->left);
    postorderPrint(root->right);
    cout << root->data << " ";
}

int main() {
    vector<int> preorder = {4, 7, 9, 6, 3, 2, 5, 8, 1};
    vector<int> inorder  = {7, 6, 9, 3, 4, 5, 8, 2, 1};

    Node* root = buildTree(preorder, inorder);

    cout << "Preorder check: ";
    preorderPrint(root);
    cout << endl;

    cout << "Inorder check: ";
    inorderPrint(root);
    cout << endl;

    cout << "Postorder of generated tree: ";
    postorderPrint(root);
    cout << endl;

    return 0;
}
```

Part I: Code Explanation

34. Node Structure

```
struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) {
```

```
        data = val;
        left = nullptr;
        right = nullptr;
    }
};
```

This creates a binary tree node.

Each node stores:

Part	Meaning
data	The value inside the node.
left	Address of the left child.
right	Address of the right child.

When a new node is created:

```
left = nullptr;
right = nullptr;
```

This means:

```
The node has no children yet.
```

---

35. The buildTree Function

```
Node* buildTree(const vector<int>& preorder, const vector<int>& inorder)
```

This is the main function the user calls.

It receives:

```
preorder array
inorder array
```

It returns:

```
address of the root node of the rebuilt tree
```

---

36. Why We Use unordered_map

The code uses:

```
unordered_map<int, int> inorderIndex;
```

This stores the position of each value in the inorder array.

Example:

```
Inorder: 7, 6, 9, 3, 4, 5, 8, 2, 1
```

The map stores:

```
7 -> 0
6 -> 1
9 -> 2
3 -> 3
4 -> 4
5 -> 5
8 -> 6
2 -> 7
1 -> 8
```

Why is this useful?

Because every time we choose a root, we need to find its position in inorder.

Without a map, we must search using a loop.

With a map, we can find the position quickly.

---

37. Building the Map

```
for (int i = 0; i < inorder.size(); i++) {
    inorderIndex[inorder[i]] = i;
}
```

This loop means:

```
For every value in inorder, remember its index.
```

Example:

```
inorderIndex[4] = 4
```

This means value 4 is at index 4 in the inorder array.

---

38. preIndex

```
int preIndex = 0;
```

This variable tells us which preorder value should be used next as the root.

At the beginning:

```
preIndex = 0
```

So we use:

```
preorder[0]
```

After using one root, we move forward:

```
preIndex++;
```

So preorder is read from left to right.

---

39. Why preIndex Is Passed by Reference

The helper function receives:

```
int& preIndex
```

The `&` means pass by reference.

Simple meaning:

```
All recursive calls share the same preIndex variable.
```

This is important because preorder must be read continuously from left to right.

If we do not pass by reference, each recursive call may receive a copy and the index may not update correctly.

Memory trick:

```
preIndex must keep moving forward.
Do not reset it inside recursion.
```

---

40. Helper Function Parameters

```
Node* buildTreeHelper(
    const vector<int>& preorder,
    int& preIndex,
    int inStart,
    int inEnd,
    unordered_map<int, int>& inorderIndex
)
```

Meaning of each parameter:

Parameter	Meaning
preorder	The preorder array.
preIndex	The current root position in preorder.
inStart	Starting index of the current inorder range.

Parameter	Meaning
inEnd	Ending index of the current inorder range.
inorderIndex	Map that quickly finds a value's inorder position.

The current recursive call only builds the subtree inside this range:

```
inStart to inEnd
```

---

41. Base Case in Code

```
if (inStart > inEnd) {
    return nullptr;
}
```

This means:

```
The current inorder range is empty.
There is no subtree here.
Return nullptr.
```

Example:

If a node has no left child, the recursive call for its left subtree gets an empty range and returns nullptr.

---

42. Choosing the Root

```
int rootValue = preorder[preIndex];
preIndex++;
```

Since preorder is:

```
Root, Left, Right
```

The next unused preorder value is always the root of the current subtree.

Then we move `preIndex` forward so the next recursive call can use the next root.

43. Creating the Root Node

```
Node* root = new Node(rootValue);
```

This creates a real node in memory.

Example:

```
rootValue = 4
```

Then:

```
Node* root = new Node(4);
```

This creates a node containing 4.

---

44. Finding the Split Point

```
int split = inorderIndex[rootValue];
```

This finds where the root is located in inorder.

That position is called the split point.

Values before the split belong to the left subtree.

Values after the split belong to the right subtree.

Example:

```
Inorder: 7, 6, 9, 3, 4, 5, 8, 2, 1
Root: 4
```

The split index is 4.

So:

```
Left side  = indices 0 to 3
Right side = indices 5 to 8
```

---

45. Building the Left Subtree

```
root->left = buildTreeHelper(
    preorder,
    preIndex,
    inStart,
    split - 1,
    inorderIndex
);
```

This means:

```
Build the left subtree from the left part of inorder.
Attach the returned subtree to root->left.
```

The left inorder range is:

```
inStart to split - 1
```

---

46. Building the Right Subtree

```
root->right = buildTreeHelper(
    preorder,
    preIndex,
    split + 1,
    inEnd,
    inorderIndex
);
```

This means:

```
Build the right subtree from the right part of inorder.
Attach the returned subtree to root->right.
```

The right inorder range is:

```
split + 1 to inEnd
```

---

47. Returning the Root

```
return root;
```

After building the left and right subtrees, we return the root of the current subtree.

This lets the previous recursive call connect it as a child.

---

48. Traversal Print Functions

The code includes these functions:

```
void preorderPrint(Node* root)
void inorderPrint(Node* root)
void postorderPrint(Node* root)
```

These are used to verify that the tree was built correctly.

If the generated tree is correct:

```
preorderPrint should match the original preorder input.
inorderPrint should match the original inorder input.
```

Part J: Inorder + Postorder Reconstruction

49. Can We Build a Tree from Inorder + Postorder?

Yes.

If node values are unique, inorder + postorder can also reconstruct a unique binary tree.

Remember:

```
Postorder = Left, Right, Root
```

So the root is at the end of postorder.

---

50. Rule for Inorder + Postorder

Use this rule:

```
Postorder gives the root from the end.
Inorder splits left and right.
```

Steps:

1. Take the last unused value from postorder.
2. That value is the root.
3. Find the root inside inorder.
4. Values left of the root in inorder belong to the left subtree.
5. Values right of the root in inorder belong to the right subtree.
6. Recursively build the subtrees.

---

51. Difference Between Preorder + Inorder and Inorder + Postorder

Combination	Where root comes from	How inorder is used
Preorder + Inorder	First unused preorder value	Split left and right
Inorder + Postorder	Last unused postorder value	Split left and right

Memory trick:

```
Preorder root is at the front.
Postorder root is at the end.
Inorder is always the splitter.
```

---

52. Important Note About Building Right First in Postorder

When using postorder from the end, the order is reversed.

Postorder is:

```
Left, Right, Root
```

If we read it backward, we get:

```
Root, Right, Left
```

So when building from postorder backward, we usually build:

```
Root first
Right subtree second
Left subtree third
```

This is different from preorder + inorder, where we usually build left first.

Part K: Complexity

53. Time Complexity Without a Map

If we do not use a map, then each time we find a root, we may search the inorder array using a loop.

For N nodes, this can become:

```
O(N²)
```

Why?

Because:

```
For each node, searching inorder may take O(N) time.
There are N nodes.
So total can become O(N * N) = O(N²).
```

---

54. Time Complexity With a Map

If we use:

```
unordered_map<int, int> inorderIndex;
```

Then finding the root position in inorder is fast.

Each node is processed once.

So the time complexity becomes:

```
O(N)
```

This is better.

---

55. Space Complexity

The algorithm uses space for:

1. The recursion stack.
2. The unordered map.
3. The newly created tree nodes.

If we only count extra helper space, the map uses:

```
O(N)
```

The recursion stack uses:

```
O(H)
```

where H is the height of the tree.

In the worst case, if the tree is skewed:

```
H = N
```

So worst-case recursion stack space is:

```
O(N)
```

---

56. Complexity Summary Table

Version	Time	Why
Basic version with linear search	$O(N^2)$	Each root may search through inorder.
Optimized version with map	O(N)	Each node is processed once.
Map space	O(N)	Stores each inorder value and index.
Recursion stack space	O(H)	H is the tree height.
Worst-case recursion stack	O(N)	Happens when the tree is skewed.

Part L: Assumptions and Limitations

57. Node Values Should Be Unique

The normal reconstruction algorithm assumes all node values are unique.

Why?

Because we must find the root inside inorder.

If values are repeated, the root value may appear more than once.

Example:

```
Inorder: 5, 5, 5
```

If root is 5, which 5 should we split around?

It is unclear.

So the beginner version assumes:

```
All node values are unique.
```

---

58. Arrays Must Match

The preorder and inorder arrays must contain the same values.

Example of valid input:

```
Preorder: A B C
Inorder:  B A C
```

Both contain:

```
A, B, C
```

Example of invalid input:

```
Preorder: A B C
Inorder:  B A D
```

These do not match because preorder has C, but inorder has D.

A valid reconstruction needs matching values.

---

59. Empty Tree Case

If the traversal arrays are empty, the tree is empty.

Example:

```
Preorder: empty
Inorder:  empty
```

Then:

```
root = nullptr
```

Part M: Common Beginner Mistakes

60. Mistake 1: Trying to Build from One Traversal Only

Wrong idea:

```
I have preorder, so I can always rebuild the tree.
```

Correct idea:

```
One traversal is not enough for an ordinary binary tree.
```

Example:

```
Preorder: A B
```

Could be:

```
A
/
B
```

or:

```
A

  B
```

So one traversal is not enough.

---

61. Mistake 2: Forgetting That Inorder Splits the Tree

The most important role of inorder is splitting.

Correct idea:

```
Find root in inorder.
Left side becomes left subtree.
Right side becomes right subtree.
```

If you forget this, reconstruction becomes confusing.

---

62. Mistake 3: Resetting preIndex

Wrong:

```
int preIndex = 0;
```

inside every recursive call.

Why is this wrong?

Because preorder must be read continuously.

Correct:

```
int& preIndex
```

This allows all recursive calls to share the same preorder index.

---

63. Mistake 4: Searching Inorder Again and Again

A beginner may use a loop every time to find root in inorder.

This works, but it can be slow.

Better:

```
unordered_map<int, int> inorderIndex;
```

This makes root lookup faster.

---

64. Mistake 5: Confusing Preorder and Inorder Roles

Wrong thinking:

```
Inorder gives the root first.
```

Correct thinking:

```
Preorder gives the root first.
Inorder splits left and right.
```

---

65. Mistake 6: Confusing Postorder Root Position

Wrong thinking:

```
Postorder gives the root first.
```

Correct thinking:

```
Postorder gives the root last.
```

So if using postorder, start from the end.

---

66. Mistake 7: Ignoring Duplicate Values

The beginner algorithm assumes unique values.

If values repeat, reconstruction may become ambiguous.

Example:

```
Preorder: 5 5 5
Inorder:  5 5 5
```

It is not clear which 5 is the root position in inorder.

So for this phase:

```
Assume all values are unique.
```

Part N: Final Summary

67. What You Learned in Phase 10

You learned that:

1. A single traversal is not enough to rebuild a unique ordinary binary tree.
2. Preorder alone is not enough.
3. Inorder alone is not enough.
4. Postorder alone is not enough.
5. Preorder + postorder is usually not enough for ordinary binary trees.
6. Inorder is important because it splits the tree into left and right subtrees.
7. Preorder + inorder can rebuild a unique tree if values are unique.
8. Inorder + postorder can rebuild a unique tree if values are unique.
9. Recursion is used to build root, left subtree, and right subtree.
10. A map can improve the time complexity from $O(N^2)$ to $O(N)$.

---

68. Final Memory Tricks

```
One traversal is not enough.
```

```
Inorder is the splitter.
```

```
Preorder gives the root first.
```

```
Postorder gives the root last.
```

```
Preorder + Inorder = build tree from front root + split.
```

```
Inorder + Postorder = build tree from end root + split.
```

Most important rule:

```
Preorder tells WHO the root is.
Inorder tells WHERE the left and right subtrees are.
```

For postorder:

```
Postorder tells WHO the root is from the end.
Inorder tells WHERE the left and right subtrees are.
```

Part O: Practice Questions

69. Practice Question 1

Given:

```
Preorder: A B C
Inorder:  B A C
```

Build the tree.

Answer:

```
   A
  /
 B   C
```

---

70. Practice Question 2

Given:

```
Preorder: A B D E C
Inorder:  D B E A C
```

Build the tree.

Answer:

```
      A
     /
    B   C
   /
  D   E
```

71. Practice Question 3

Can preorder alone rebuild a unique binary tree?

Answer:

```
No.
```

Reason:

```
Preorder gives root order, but it does not always show whether a child is on the
left or right.
```

---

72. Practice Question 4

Why is inorder important?

Answer:

```
Because inorder separates the left subtree from the right subtree once the root
is known.
```

---

73. Practice Question 5

What is the time complexity of the optimized algorithm using a map?

Answer:

```
O(N)
```

Reason:

```
Each node is processed once, and each root position is found quickly using the
map.
```

---

74. Final Conclusion

Phase 10 completes the tree curriculum.

You started by learning what a tree is. Then you learned binary trees, formulas, representation, traversals, recursion, strict trees, complete trees, and M-ary trees.

Now you can use traversal arrays to rebuild a binary tree.

The most important beginner idea is:

```
Use preorder or postorder to find the root.
Use inorder to split the tree into left and right subtrees.
Repeat recursively until the tree is complete.
```

That is the heart of generating trees from traversals.